

秋冬季训练营三阶段(7) 组件化内核设计与实践

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

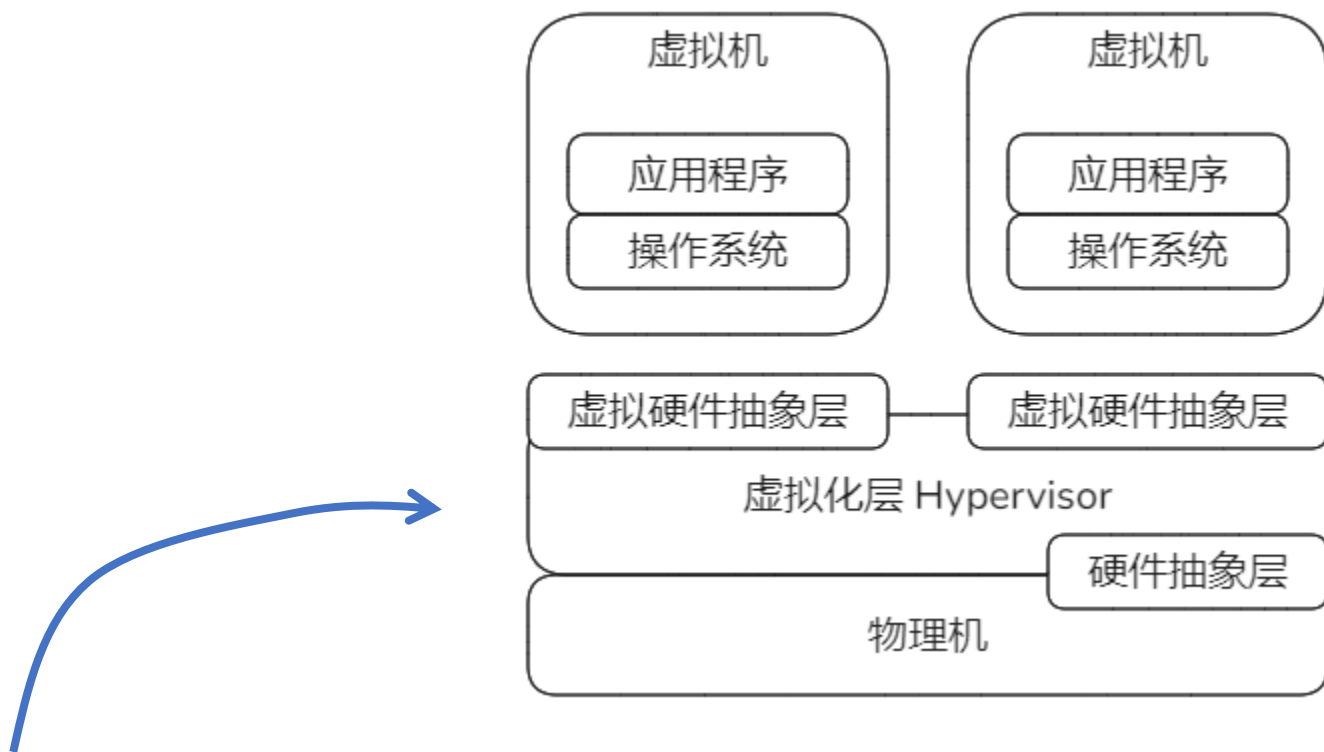
2024.11.25

第三部分

组件化扩展 - Hypervisor

虚拟化和Hypervisor概念

基于一台物理计算机系统建立一台或多台虚拟计算机系统（虚拟机），每台虚拟机拥有自己的独立的虚拟机硬件，支持一个独立的虚拟执行环境。每个虚拟机中运行的操作系统，认为自己独占该执行环境，无法区分该环境是物理机还是虚拟机。

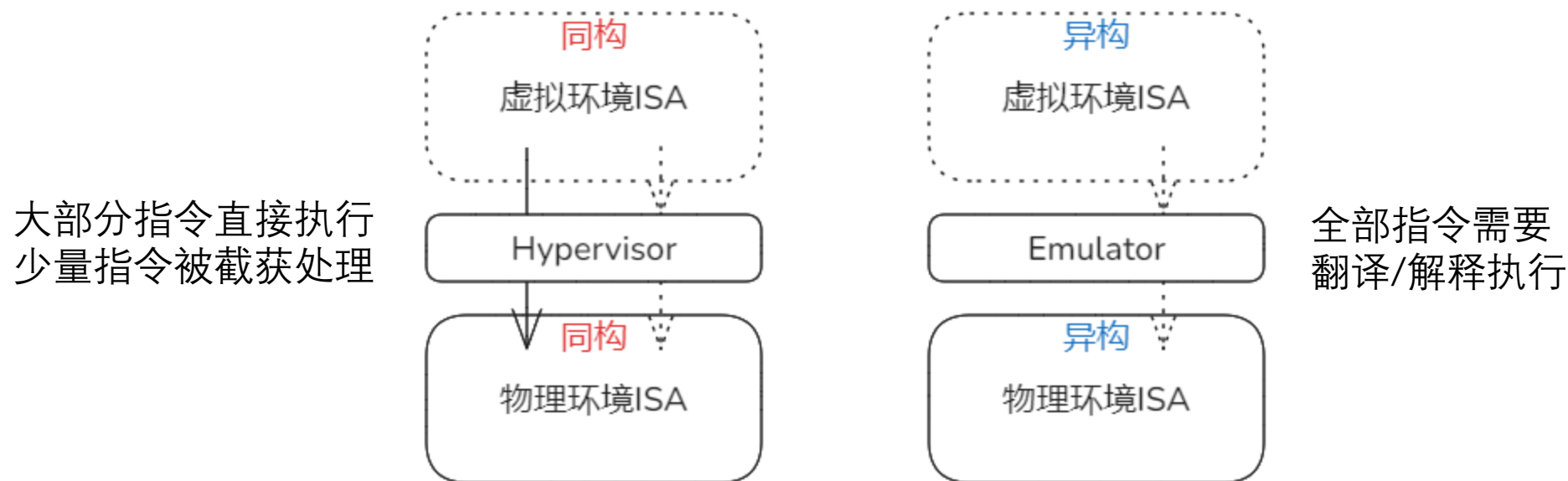


Hypervisor即图中的虚拟化层软件：

运行在物理服务器和操作系统之间的中间软件层，允许多个操作系统和应用共享硬件。

Hypervisor与模拟器Emulator的区别

根本区别：虚拟运行环境和支撑它的物理运行环境的体系结构即ISA是否一致。



根据1974年，Popek和Goldberg对虚拟机的定义：

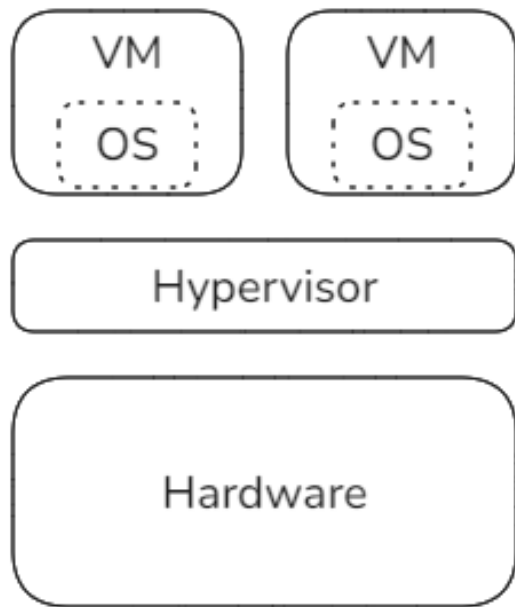
虚拟机可以看作是物理机的一种高效隔离的复制，蕴含三层含义：同质、高效和资源受控。

同质要求ISA的同构，高效要求虚拟化消耗可忽略，资源受控要求中间层对物理资源的完全控制。

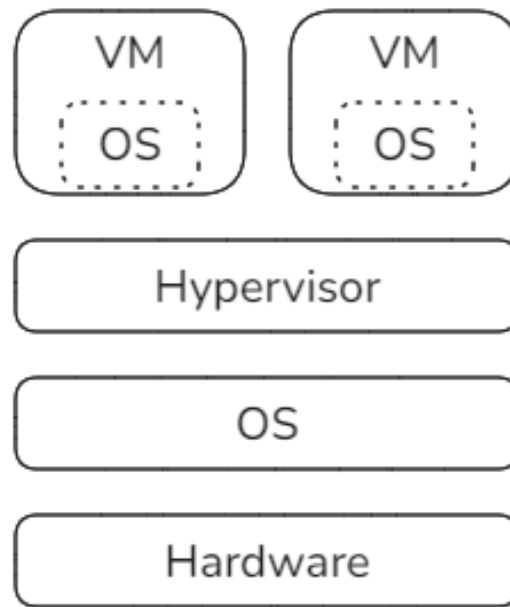
Hypervisor必须符合上述要求，而模拟器更侧重的是仿真效果，对性能效率通常没有硬性要求。

虚拟化类型

主要包括两种类型：区别在于构成的层级、引导的过程、实现的基础。



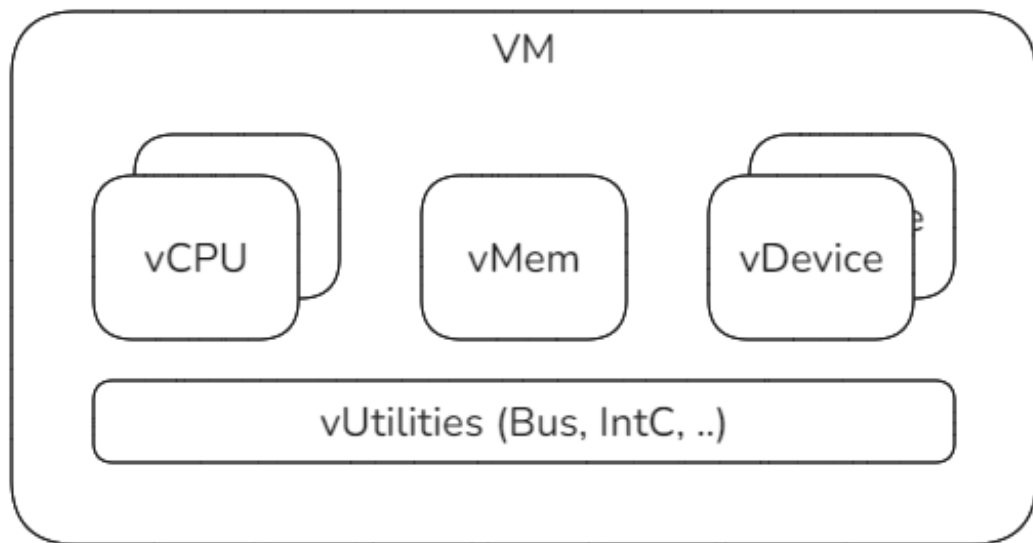
I型：直接在硬件平台运行



II型：在宿主OS之上运行

我们目前的实验仅针对I型虚拟化。

Hypervisor支撑的资源对象层次



Hypervisor

Host

VM: 管理地址空间；同一整合下级各类资源

vCPU: 计算资源虚拟化，VM中执行流

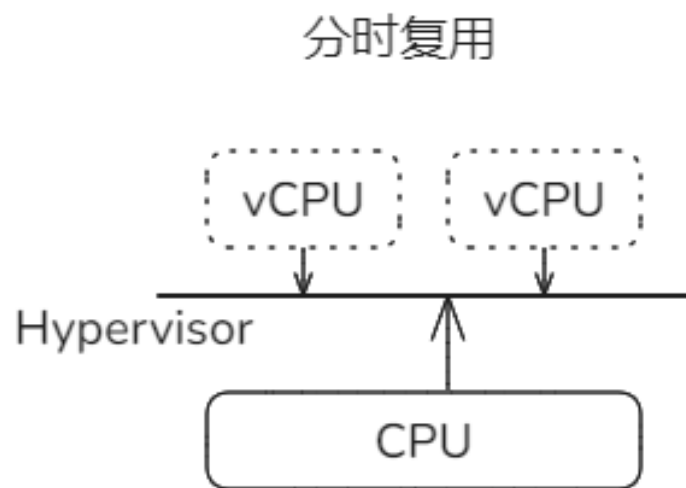
vMem: 内存虚拟化，按照VM的物理空间布局

vDevice: 设备虚拟化：包括直接映射和模拟

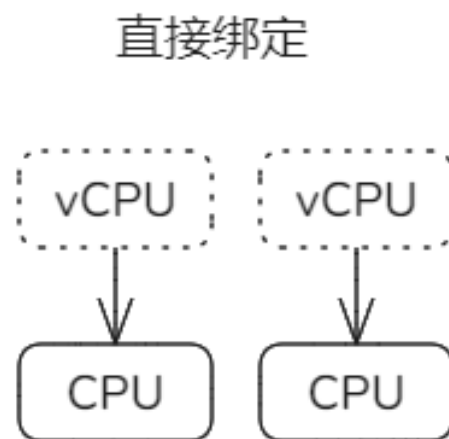
vUtilities: 中断虚拟化、总线发现设备等

Hypervisor模拟了完整的物理机环境，物理环境下的资源在虚拟环境下都有对应。

vCPU与CPU的关系



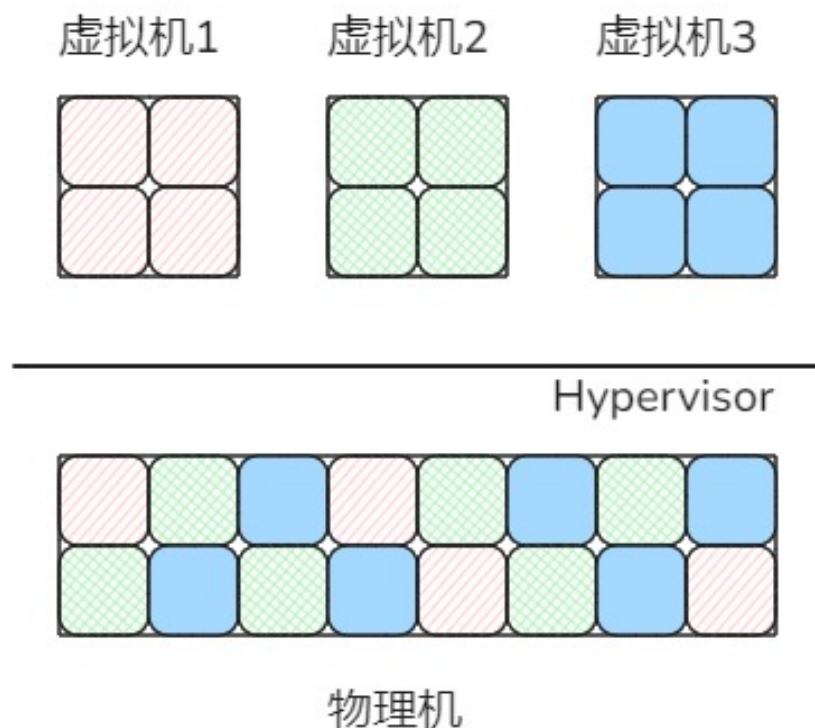
通常方法：效率较低
Hypervisor建立多任务
对应多个vCPU



效率较高，vCPU数量受限

本系列实验基于直接绑定模式，因为其实现简单

虚拟内存与物理内存的关系



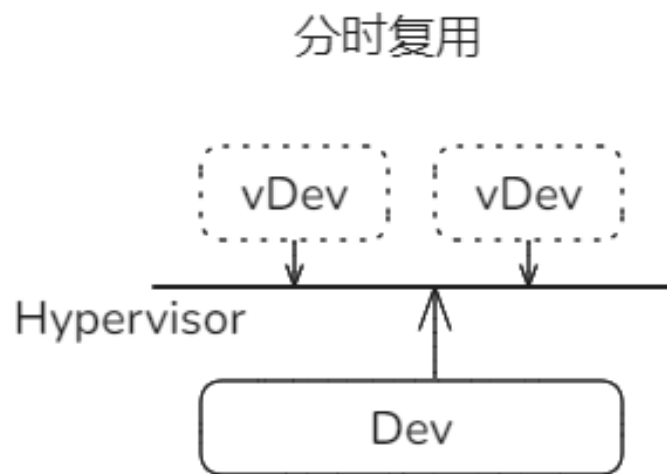
从每个虚拟机的角度，它们的物理内存是连续的。实际它们并不拥有实际的物理内存，只是假象。

从物理机的角度，分属各虚拟机的内存页是不连续的，间隔零散的。

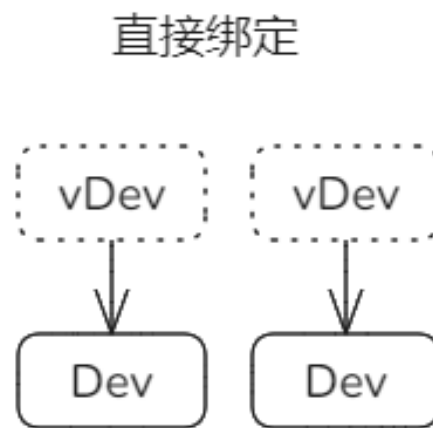
Hypervisor基于页表扩展等机制建立和维护二者之间的对应关系。

虚拟设备与物理设备的关系

类似于CPU的情况，虚拟设备与物理设备的对应关系也是两种。



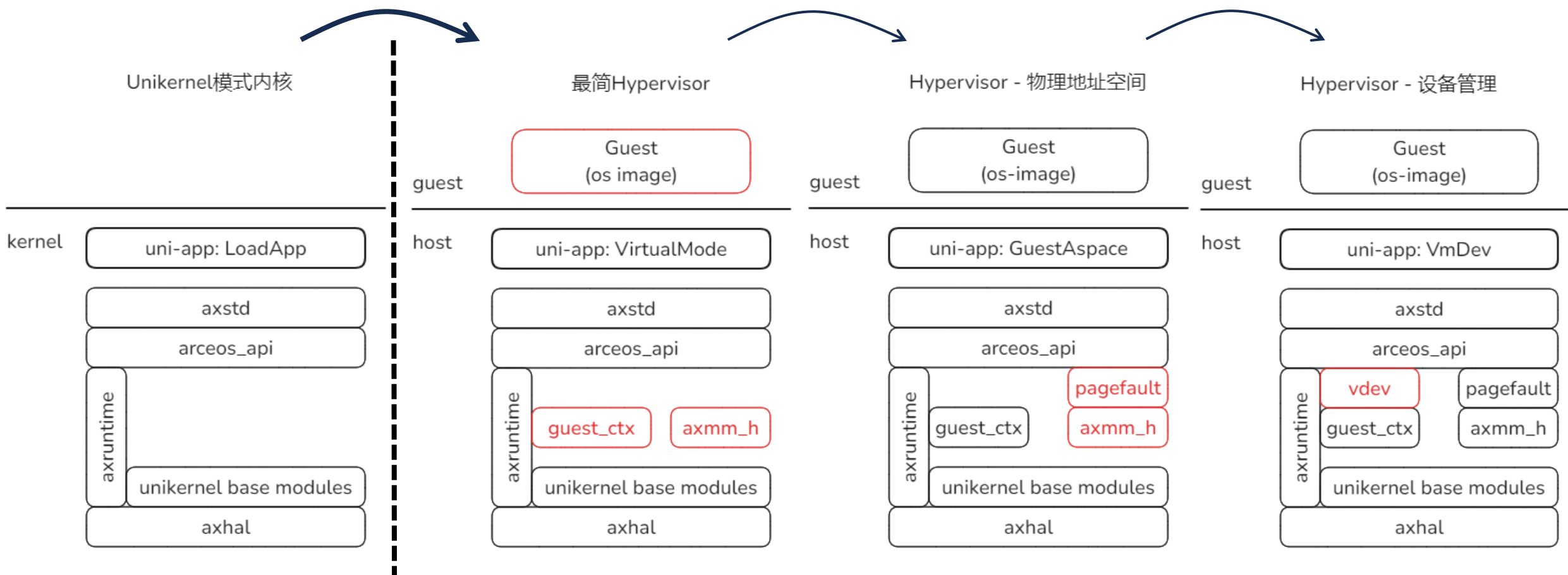
例如物理网卡可以作为网桥被多个虚拟网卡分时复用。



例如物理网卡可以与虚拟网卡一一绑定。

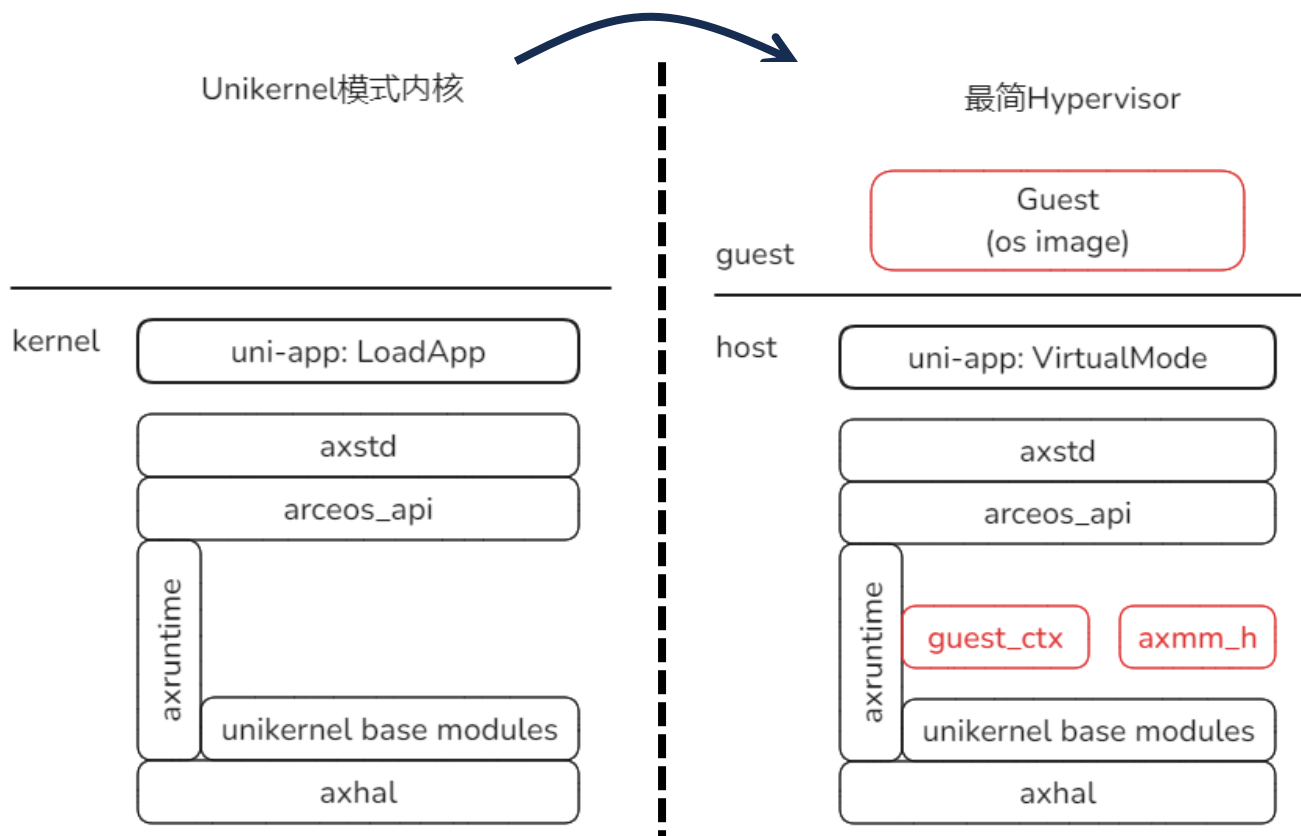
第三部分 Hypervisor

跨越模式边界



H.1.0 VirtualMode

跨越模式边界



实验命令行:

make payload

./update_disk.sh payload/skernel/skernel

make run A=tour/h_1_0/ BLK=y

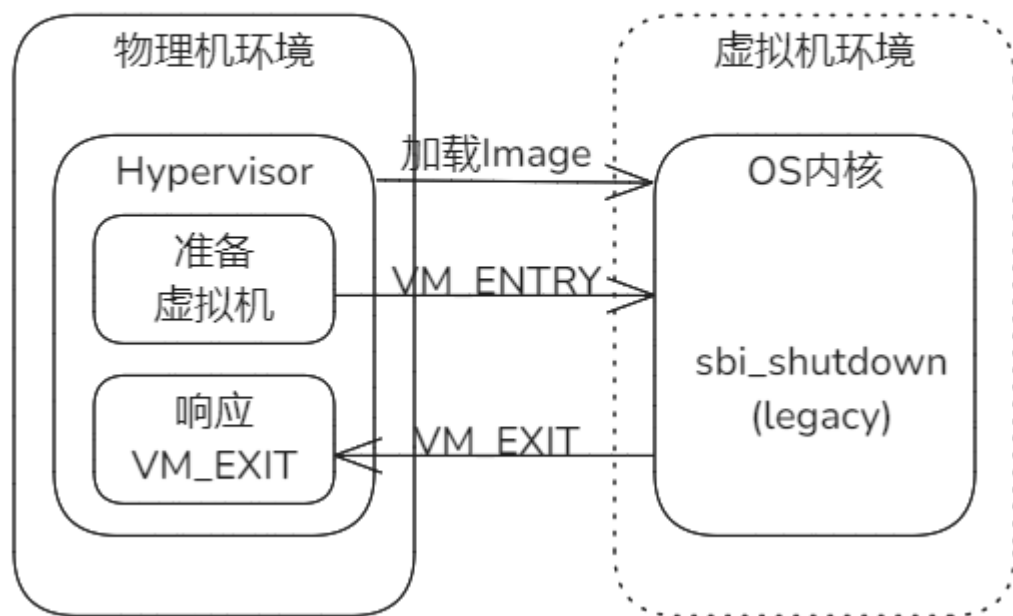
本节目标:

1. Hypervisor实现的基础
2. 虚拟化模式切换机制概述

h_1_0实验目标和需要解决的问题

实验h_1_0建立最简化的Hypervisor，用于分析其原理。

实验 h_1_0



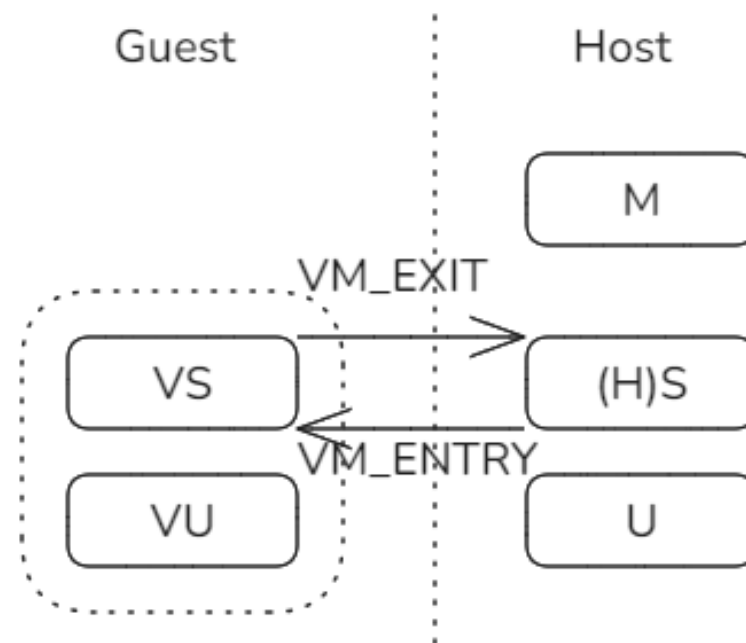
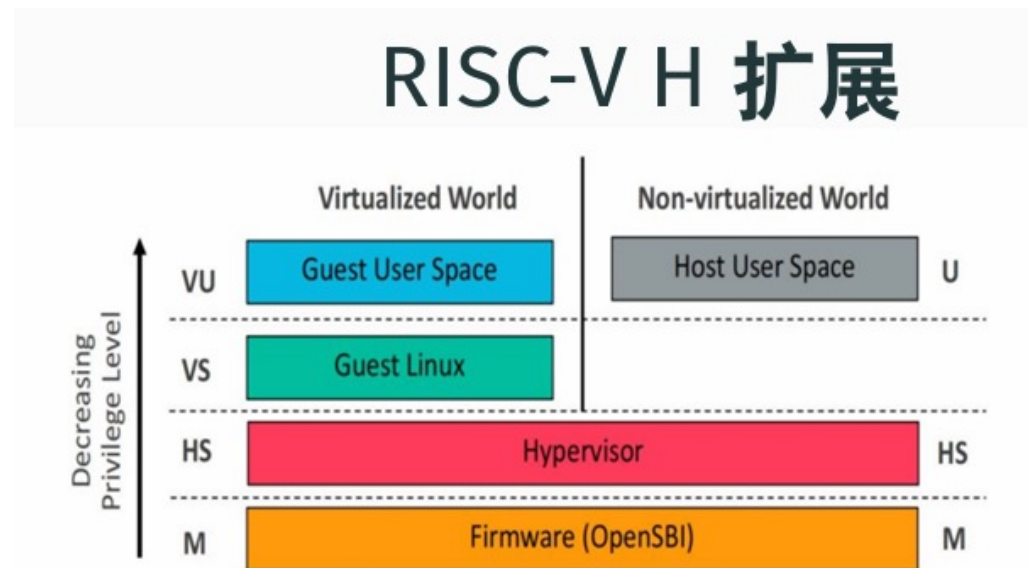
最简Hypervisor执行流程：

1. 加载Guest OS内核Image到新建地址空间。
2. 准备虚拟机环境，设置特殊上下文。
3. 结合特殊上下文和指令sret切换到V模式，即VM-ENTRY。
4. OS内核只有一条指令，调用sbi-call的关机操作。
5. 在虚拟机中，sbi-call超出V模式权限，导致VM-EXIT退出虚拟机，切换回Hypervisor。
6. Hypervisor响应VM-EXIT的函数检查退出原因和参数，进行处理，由于是请求关机，清理虚拟机后，退出。

需要解决的问题：

1. 准备能够进入虚拟化模式的特殊上下文。
2. 为响应VM-EXIT设置处理函数。

Riscv64在特权级模式的H扩展



特权级从三个扩展到五个，新增了与Host平行的Guest域

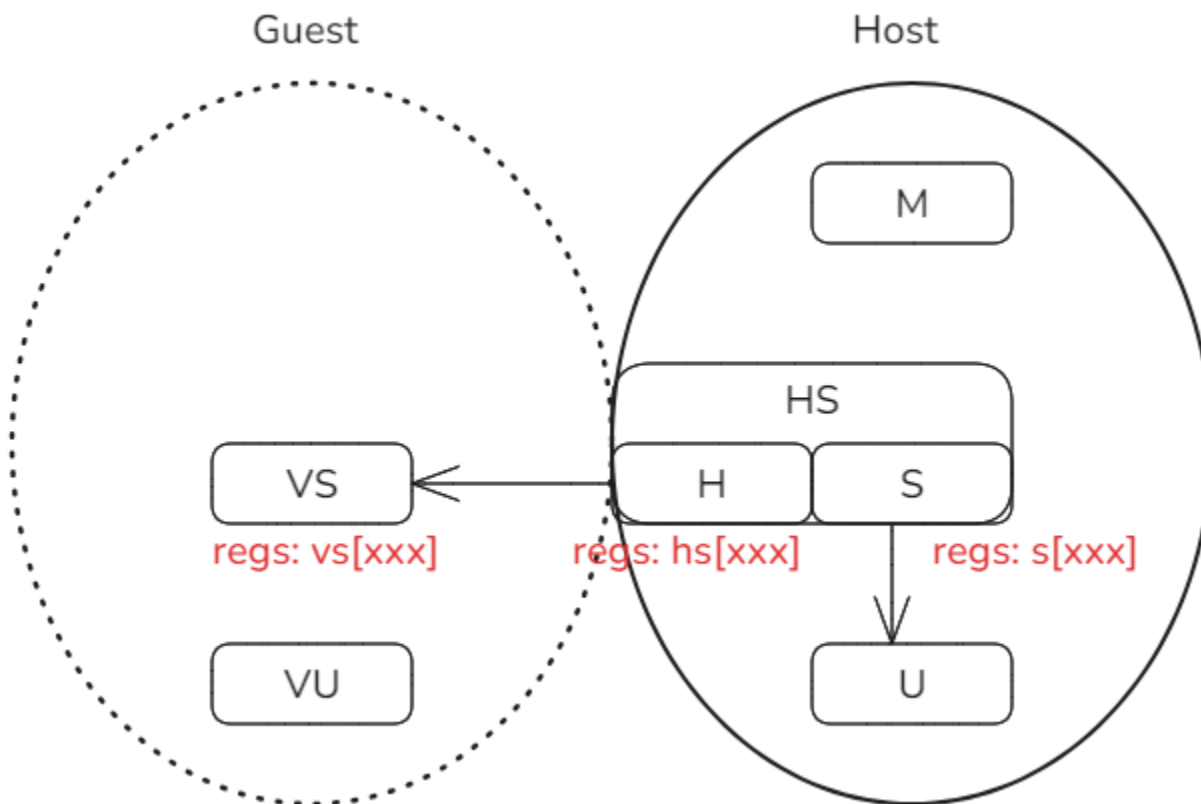
- 原来的S增强了对虚拟化支持的特性后，称它为HS。
- M/HS/U形成Host域，用来运行I型Hypervisor或者II型的HostOS，三个特权级的作用不变。
- VS/VU形成Guest域，用来运行GuestOS，这两个特权级分别对应内核态和用户态。
- HS是关键，作为联通真实世界和虚拟世界的通道。体系结构设计了双向变迁机制。

体系结构H扩展后的寄存器

H扩展后，S模式发送明显变化：原有s[xxx]寄存器组作用不变，新增hs[xxx]和vs[xxx]

hs[xxx]寄存器组的作用：面向Guest进行路径控制，例如异常/中断委托等

vs[xxx]寄存器组的作用：直接操纵Guest域中的VS，为其准备或设置状态



为进入虚拟化模式准备的条件（1）

ISA寄存器misa第7位代表Hypervisor扩展的启用/禁止。
对这一位写入0，可以禁止H扩展。

ISA指令集手册

Table 10. Encoding of Extensions field in `misa`. All bits that are reserved are read.

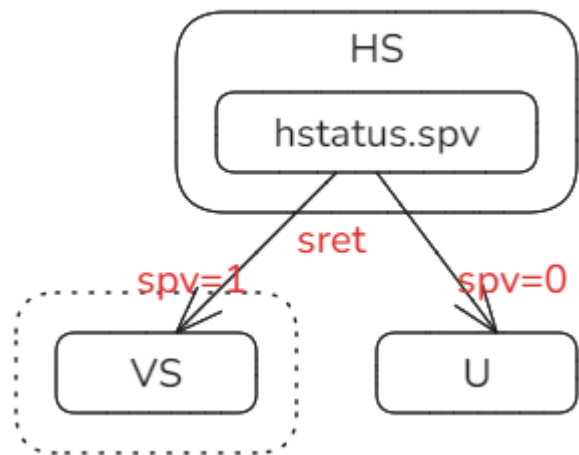
Bit	Character	Description
0	A	Atomic extension
1	B	B extension
2	C	Compressed extension
3	D	Double-precision floating-point extension
4	E	RV32E/64E base ISA
5	F	Single-precision floating-point extension
6	G	Reserved
7	H	Hypervisor extension

OpenSBI启动时显示ISA扩展支持

```
Boot HART ID : 0
Boot HART Domain : root
Boot HART Priv Version : v1.12
Boot HART Base ISA : rv64imafdc
Boot HART ISA Extensions : sstc,zicntr,zihpm,zicboz,zicbom
Boot HART PMP Count : 16
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits: 54
Boot HART MHPM Info : 16 (0x0007fff8)
Boot HART MIDELEG : 0x0000000000001666
Boot HART MEDELEG : 0x0000000000f0b509
```

为进入虚拟化模式准备的条件（2）

进入V模式路径的控制：**hstatus第7位**SPV记录上一次进入HS特权级前的模式，1代表来自虚拟化模式。执行sret时，根据SPV决定是返回用户态，还是返回虚拟化模式。



ISA指令集手册H扩展

19	18	17	12	11	10	9	8	7	6	5	4	0
WPRI		VGEIN[5:0]		WPRI		HU	SPVP	SPV	GVA	VSBE	WPRI	
2		6		2		1	1	1	1	1	5	

Figure 70. Hypervisor status register (hstatus) when HSXLEN=64.

```
// Set Guest bit in order to return to guest m
hstatus.modify(hstatus::spv::Guest);
// Set SPVP bit in order to accessing VS-mode
hstatus.modify(hstatus::spvp::Supervisor);
```

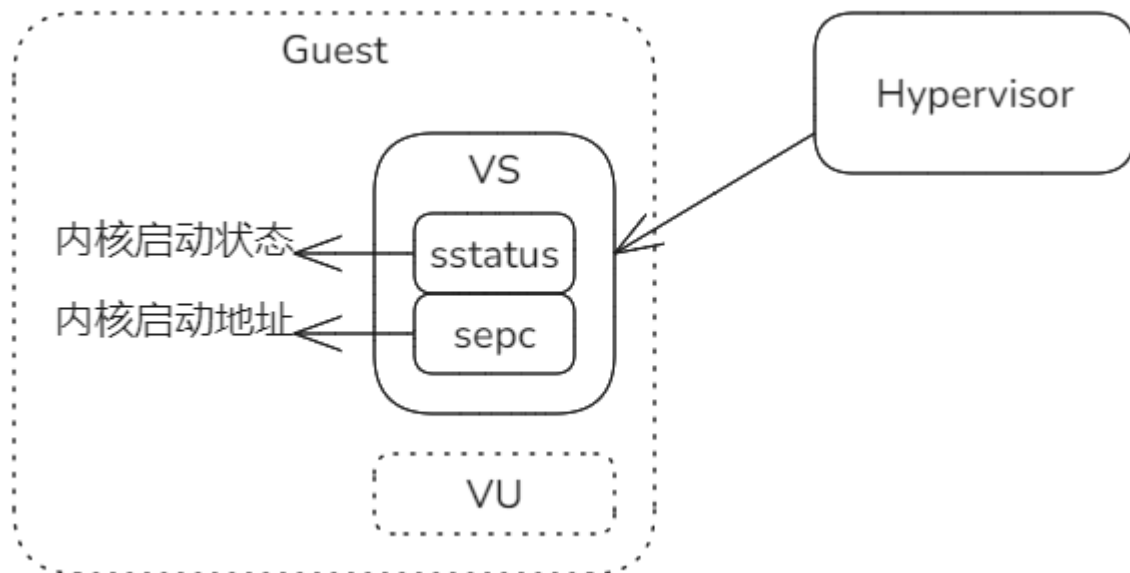
SPV指示特权级模式的来源；
SPVP指示HS对V模式下地址空间是否有操作权限，1表示有权限操作，0无权限。

为进入虚拟化模式准备的条件 (3)

Hypervisor首次启动Guest的内核之前，伪造上下文作准备：

设置Guest的sstatus，让其初始特权级为Supervisor；

设置Guest的sepc为OS启动入口地址VM_ENTRY，VM_ENTRY值就是内核启动的入口地址，对于Riscv64，其地址是0x8020_0000。

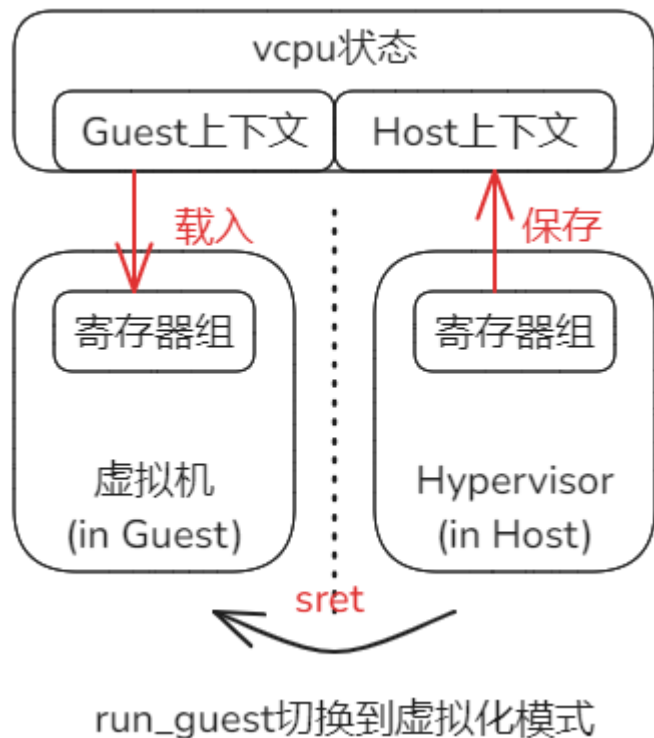


```
// Set sstatus in guest mode.  
let mut sstatus = sstatus::read();  
sstatus.set_spp(sstatus::SPP::Supervisor);  
ctx.guest_regs.sstatus = sstatus.bits();  
// Return to entry to start vm.  
ctx.guest_regs.sepc = VM_ENTRY;
```

从Host到Guest的切换run_guest

每个vCPU维护一组上下文状态，分别对应Host和Guest。

从Hypervisor切换到虚拟机时，暂存Host中部分可能被破坏的状态；加载Guest状态；然后执行sret完成切换。封装该过程的专门函数run_guest。



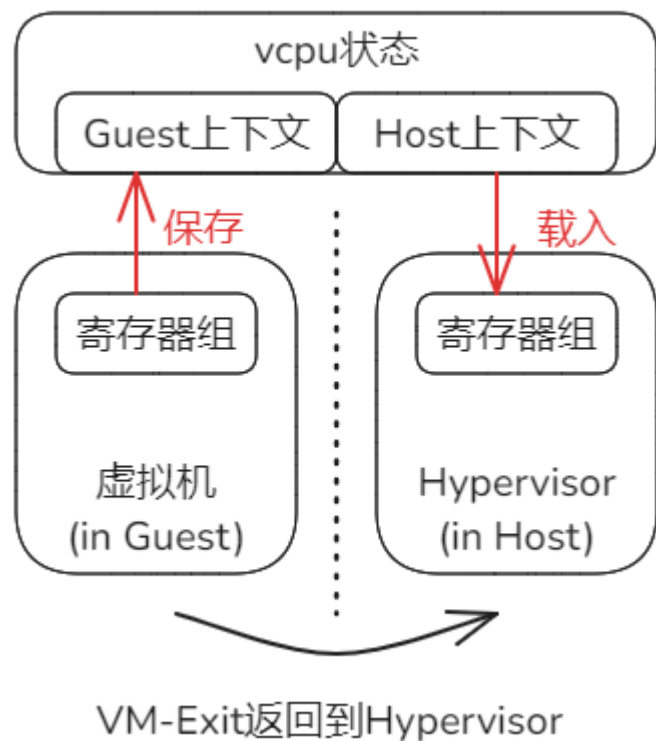
```
/// Enter the guest given in `VmCpuRegisters` from `a0`
.global _run_guest
_run_guest:
    /* Save hypervisor state */

    /* Save hypervisor GPRs (except T0-T6 and a0, which
sd    ra, ({hyp_ra})(a0)
sd    gp, ({hyp_gp})(a0)
sd    tp, ({hyp_tp})(a0)
sd    s0, ({hyp_s0})(a0)
sd    s1, ({hyp_s1})(a0)
sd    a1, ({hyp_a1})(a0)
sd    a2, ({hyp_a2})(a0)
sd    a3, ({hyp_a3})(a0)
sd    a4, ({hyp_a4})(a0)
sd    a5, ({hyp_a5})(a0)
sd    a6, ({hyp_a6})(a0)
sd    a7, ({hyp_a7})(a0)
sd    s2, ({hyp_s2})(a0)
```

run_guest切换过程的细节较多，下节课单独介绍。

因为VM-Exit从Guest到返回Host

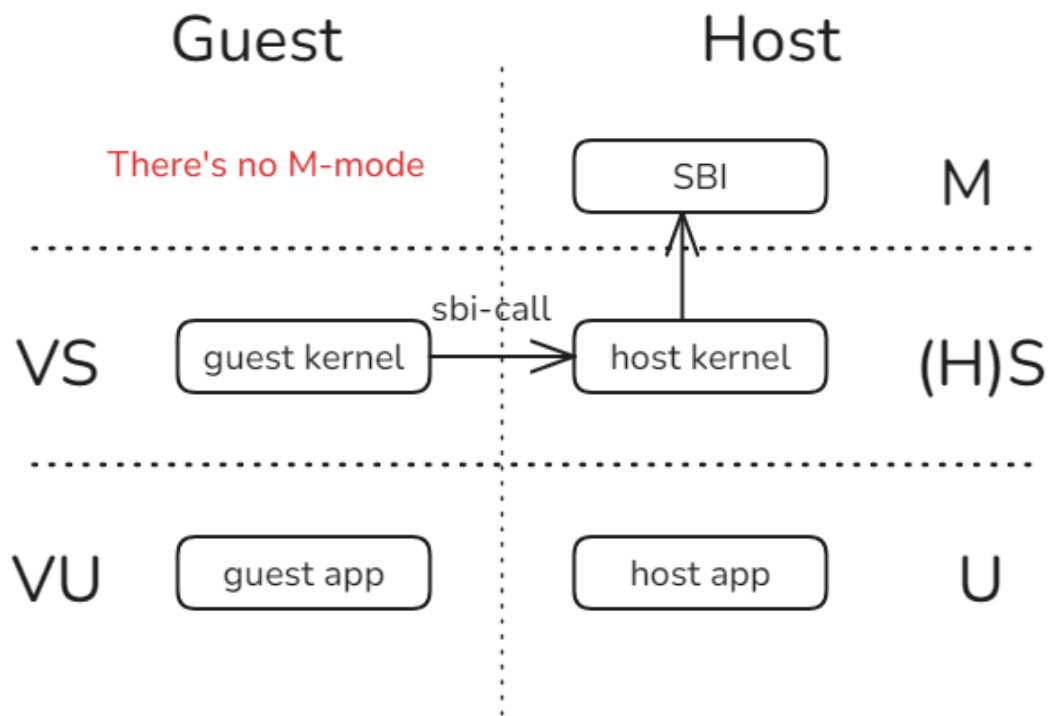
执行上页从Host到Guest的逆过程，具体机制包含在run_guest之中。



返回过程包含在run_guest过程中，
具体实现机制下节课与上页一起介绍。

VM-Exit原因1: 执行特权操作

在Guest虚拟环境下，不存在M模式，没有OpenSBI。因而sbi-call超出当前环境的处理能力，触发模式切换，回到Host模式的Hypervisor中处理。

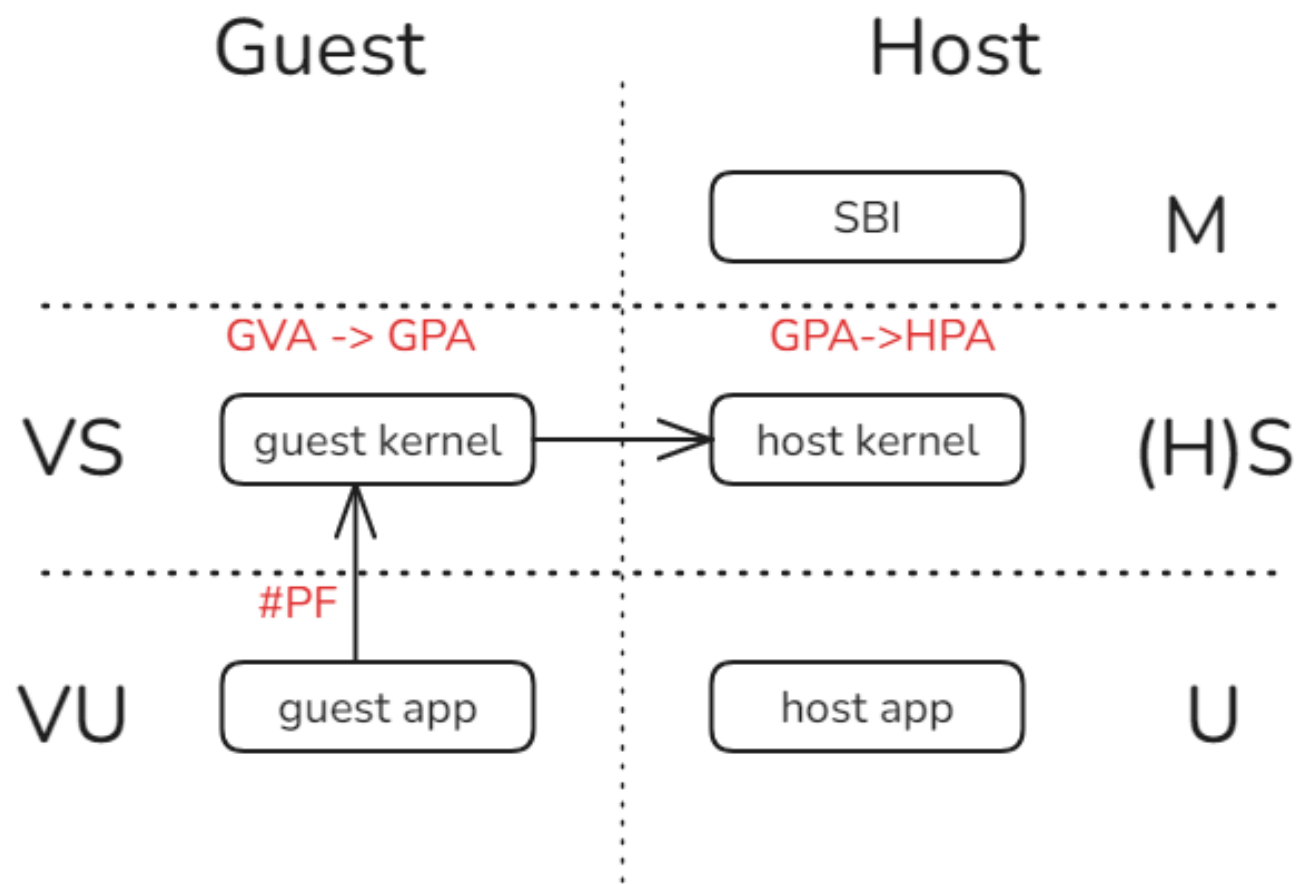


```
match scause.cause() {  
  Trap::Exception(Exception::VirtualSupervisorEnvCall) => {  
    let sbi_msg = SbiMessage::from_regs(ctx.guest_regs.gpr  
    ax_println!("VmExit Reason: VSuperEcall: {:?}", sbi_ms  
    if let Some(msg) = sbi_msg {  
      match msg {  
        SbiMessage::Reset(_) => {  
          ax_println!("Shutdown vm normally!");  
        },  
        _ => todo!(),  
      }  
    } else {  
      panic!("bad sbi message! ");  
    }  
  },  
}
```

在h_1_0示例中，仅需要处理SBI-Call的Shutdown请求。

VM-Exit原因2: 嵌套的#PF

Guest环境的物理地址区间尚未映射，导致二次缺页，切换进入Host环境。
下节课内容。



课后练习

[simple_hv]: 实现最简单的Hypervisor，响应VM_EXIT常见情况。

准备：（如果看不到**exercises/simple_hv**或**skernel2**，回到main分支执行**git pull** 更新工程）

执行如下三行，触发错误，分析原因。

```
make payload
```

```
./update_disk.sh payload/skernel2/skernel2
```

```
make run A=exercises/simple_hv/ BLK=y
```

要求：根据panic指示修改实现，使测例通过。预期输出：

```
Hypervisor ...
app: /sbin/skernel2
paddr: PA:0x80642000
Bad instruction: 0xf14025f3 sepc: 0x80200000
LoadGuestPageFault: stval0x40 sepc: 0x80200004
VmExit Reason: VSuperEcall: Some(Reset(Reset { reset_type: Shutdown, reason: NoReason }))
a0 = 0x6688, a1 = 0x1234
Shutdown vm normally!
[ 0.330726 0:2 axruntime::lang_items:5] panicked at exercises/simple_hv/src/main.rs:56:5:
Hypervisor ok!
cloud@server:~/gitWork/oscamp/arceos$
```

提示：需要处理两个VM-Exit原因，可以对照查看payload/skernel2/中Guest OS内核的实现。注意sepc寄存器可能需要调整偏移，否则会异常卡死；另外a0和a1应当分别在处理Exit原因时设置。

课后题（补充提示）

计算spec需要增加的偏移， 需要知道指令长度。可以通过如下命令查看编译后的指令：
riscv64-linux-gnu-objdump -D ./target/riscv64gc-unknown-none-elf/release/skernel2

Disassembly of section .text:

```
ffffffc080200000 <_start>:
ffffffc080200000:      f14025f3      csrr      a1,mhartid
ffffffc080200004:      04003503      ld       a0,64(zero) # 40 <_percpu_load_end+0x40>
ffffffc080200008:      48a1          li       a7,8
ffffffc08020000a:      00000073      ecall
...
```